Veritas

Tamper-evident marks and certificate ledger on PostgreSQL

Database Management Systems (CD252IA), RV College of Engineering | generated 24 October 2026

1. Problem and approach

Marks and certificates are changed after the fact, quietly. Veritas makes that detectable and mostly impossible **inside the database**: the rules live in triggers, constraints and row-level security, so no application bug, direct SQL session or clever client can skip them. Every mark write is an append-only, hash-chained revision; revisions are sealed into Merkle batches whose roots are pinned in an external witness; certificates store only a salted hash and verify publicly with a QR code.

18 tables | 109 columns | 30 triggers | row-level security | hash chain | Merkle tree | PostgreSQL 16 | FastAPI | React

2. Entity-relationship diagram

Generated from the live schema. Arrows point from the referencing table (N) to the referenced table (1). Pink: the tamper-evidence tables; blue: the marks workflow.



3. Relational schema

- **anchors**(anchor_id, batch_id, anchor_type, anchor_ref, anchored_at)
- **anomaly_alerts**(alert_id, revision_id, reviewed_by, model_name, score, top_features, status, created_at, reviewed_at)
- **app_credentials**(user_id, password_hash)
- **assessments**(course_id, type, max_marks, freeze_at)
- **certificate_documents**(cert_id, mime, doc, created_at)
- **certificates**(cert_id, student_id, issued_by, cert_type, doc_hash, salt, status, issued_at, revoked_at)
- **change_requests**(request_id, mark_id, requested_by, approved_by, proposed_value, reason, status, requested_at, decided_at)
- **courses**(course_id, course_code, title, credits, semester, faculty_id, institution_id)
- **faculty**(user_id, employee_id, department, can_approve)
- **institutions**(institution_id, slug, name)
- **integrity_runs**(run_id, kind, ok, detail, started_at)
- **login_attempts**(attempt_id, email, ip, ok, at)
- **mark_revisions**(revision_id, mark_id, course_id, seq, request_id, prev_revision_id, old_value, new_value, changed_by, reason, changed_at, prev_hash, row_hash)
- **marks**(mark_id, student_id, course_id, assessment_type, current_value, version, updated_at)
- **merkle_batches**(batch_id, course_id, seq_from, seq_to, merkle_root, created_at)
- **notifications**(notification_id, user_id, kind, message, link, created_at, read_at)
- **students**(user_id, usn, branch, semester)
- **users**(user_id, name, email, role, created_at, institution_id)

Underlined = primary key. The full data dictionary (types, defaults, constraints) is in `docs/data_dictionary.md`.

4. Normalisation

Relation	Normal form	Reasoning
users	BCNF	Every attribute depends only on user_id; email is a second candidate key. The role selects a disjoint specialisation (students or faculty), enforced by triggers.
students, faculty	BCNF	Specialisations of users keyed by user_id; usn and employee_id are candidate keys.
courses	BCNF	Candidate keys course_id and (course_code, semester); faculty_id is a plain foreign key.
assessments	BCNF	Weak entity keyed by (course_id, type): max_marks and freeze_at depend on the whole key.
marks	BCNF	Associative entity; candidate keys mark_id and (student_id, course_id, assessment_type); (course_id, assessment_type) references assessments.
change_requests	3NF	All attributes depend on request_id. approved_by and decided_at are filled together (check constraint).
mark_revisions	2NF-3NF with one deliberate redundancy	course_id is determined by mark_id (marks.course_id), which makes it a transitive dependency. It is kept on purpose as the chain key so each course has its own chain and lock; a trigger rejects any row whose course_id differs from the mark's.
merkle_batches, anchors	BCNF	Keyed by batch_id / anchor_id; (course_id, seq_from) is a candidate key of a batch.
certificates, certificate_documents	BCNF	Only sha256(salt || content digest) is stored, never the content; the document is write-once and keyed by cert_id.

notifications, integrity_runs, login_attempts, institutions	BCNF	Operational tables, each depending only on its surrogate key.
v_course_result, v_student_sgpa	(views)	SGPA is derived by a view and never stored, so it can never disagree with the marks (no update anomaly).

5. How the database enforces the rules

Rule	Mechanism
Revisions are append-only	BEFORE UPDATE/DELETE/TRUNCATE trigger raises on mark_revisions, merkle_batches, anchors, certificate_documents
seq, prev_hash, row_hash come from the database	BEFORE INSERT trigger on mark_revisions; a row lock on the course serialises that course's chain only
Marks change only through apply_change()	marks_guard trigger checks a transaction-local flag that only the SECURITY DEFINER function sets
Only the course faculty writes marks	Checked in the chain trigger against courses.faculty_id
Frozen assessments need an approved request	Chain trigger: now() >= freeze_at requires an approved change_request that matches the revision
Maker is not the checker	CHECK approved_by <> requested_by plus can_approve test in the update trigger
Each role sees only its own data	Row-level security policies on users, students, marks, revisions, requests, certificates, alerts, notifications, documents
Each institution sees only its own data	RESTRICTIVE policies (AND-ed with the role policies) on all 12 data tables, using owner-run lookup functions so policies never recurse
The app connects with least privilege	Role veritas_app: SELECT on tables it needs, EXECUTE on the write functions only; verify_*, seal_batch, record_anchor are owner-only
Certificates never store content	Only sha256(salt || sha256(content)) is stored; verification returns valid / revoked / unknown
Users are told what concerns them	AFTER triggers write notifications for requests, decisions, mark changes, alerts and certificates

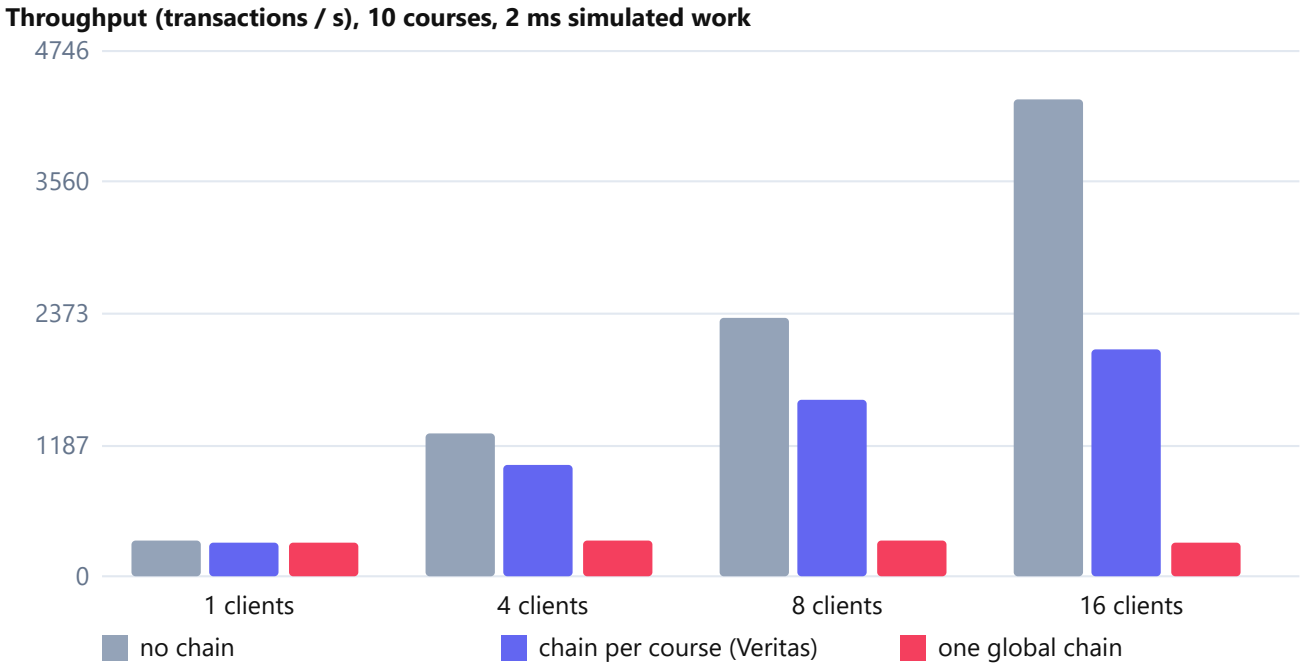
6. Tamper detection and threat model

Leaf = SHA-256('L' + row_hash); node = SHA-256('N' + left + right); the SQL `merkle_root()` and the Python `merkle.py` are cross-checked for equality. Row hashes use a canonical JSON array so free text cannot create ambiguity.

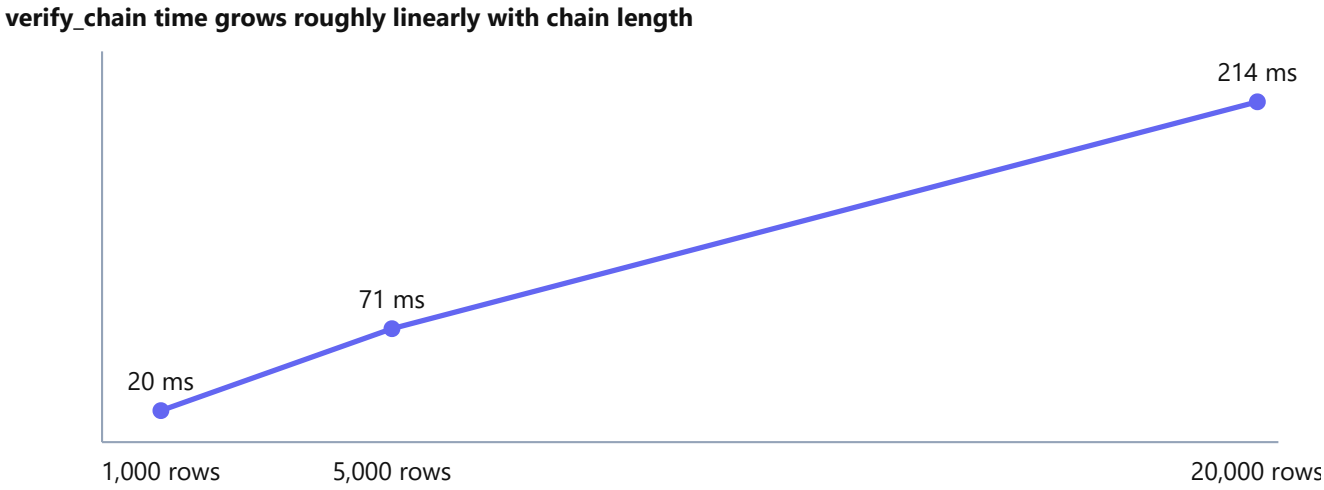
Attack	What catches it	Test
Edit a mark's value in mark_revisions	verify_chain: row_hash no longer matches	T100, P08
Edit a row and re-hash only that row	Next row's prev_hash no longer matches	T101
Delete a middle revision	Sequence gap	T102
Re-hash the whole chain after changing history	old_value continuity check; and the external witness root	T104, P08
Edit marks.current_value directly	verify_marks_current compares with the ledger	T105
Forge a stored Merkle root	verify_batches recomputes it	T106
Truncate the tail of the chain	Marks cross-check and batch check; witness reports the missing batch	T108, T109, P09
Rewrite database AND witness together	Not detectable: the witness must live outside the DBA's control (git remote)	P11 (documented negative result)
Brute-force a login	Account locks after 5 failures for 15 minutes	API A03
Cross-site request forgery	State-changing calls need the per-session CSRF token	API A04
Read another student's marks / documents	Row-level security returns nothing	T-series, AX06, AX11, API C03
Read or alter another institution's data	Restrictive RLS policies on every table; mixed-institution rows are refused by triggers	AX18-AX23, API I04-I10
Verifier races an anchor and raises a false alarm	One lock serialises witness writes and checks	API W05

7. Concurrency and performance

A single global chain would serialise every writer. Veritas chains per course, so writers on different courses never wait for each other (concurrency tests C1-C3 prove same-course writers queue and other-course writers do not).



At 16 clients a per-course chain sustains **2057 tps** against **301 tps** for one global chain, **6.8x** more.



An index on (course_id, seq) turns the "latest revision of a course" lookup from a 6.4 ms sequential scan into a 0.06 ms index scan.

8. Application features

Area	What it does
Marks workflow	Entry, optimistic version check, freeze, change request, checker approval, apply with request id; CSV bulk import and export
Integrity	Seal Merkle batches, anchor roots to a git-backed witness (push to a remote for real separation), inclusion proofs, scheduled verification, alerts to admins by notification, webhook and e-mail
Anomaly detection	IsolationForest plus rules over revision features; alerts explain why they fired and are triaged by a checker; model version is recorded
Certificates	Issue custom certificates or ledger-built transcripts; PDF with scan-to-verify QR stored write-once; public verifier works from the QR alone
Selective disclosure	A student exports a proof of one revision (fields, Merkle path, witness record); verify_disclosure.py checks it offline without revealing other marks
Security	Login throttling, per-session CSRF token, random session secret, optional HTTPS-only cookies; Caddy gives automatic TLS in the Docker stack
Operations	backup_restore.py backs up only a verified ledger and proves each restore; Docker Compose stack; GitHub Actions CI
Multi-institution	Every user and course belongs to an institution. Restrictive row-level-security policies isolate every table per institution; triggers refuse a course, faculty member or student from different institutions; admin dashboards, directory, sealing, anchoring, transcripts, alerts and notifications are scoped; the scheduler verifies each institution separately

9. Test summary

Suite	Tests
Ledger suite (SQL)	84
Application-layer suite (SQL, incl. multi-institution isolation)	25
Python cross-checks and attacks	11
Concurrency	3
API end-to-end (incl. isolation and race checks)	32
Backup / restore	6
Browser end-to-end (Chrome)	12
Total	**173**

10. Limits and future work

- The witness is only outside the administrator's reach if it is pushed to a remote they do not control; on one machine it is tamper-evident, not tamper-proof. The immudb backend is written but untested.
- Revisions made after the last anchor are not yet pinned; the scheduler shortens that window (default 15 minutes).
- Single sign-on (OIDC) is not implemented; accounts use bcrypt passwords with throttling.

- The Docker stack has not been run in the development environment (no Docker daemon).
- The witness (one file or git repo) is shared by all institutions; records carry the institution slug and each admin only verifies their own, but the file itself is common.